



UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA



UNIVERSITA' DEGLI STUDI DI PADOVA  
DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

**Elaborazione audio in tempo reale su  
microcontrollore per l'emulazione di effetti per  
strumenti musicali**

Relatore:  
PROF. CARLO FANTOZZI

Laureando:  
ALESSIO FAGNANI  
2103431

Anno Accademico: 2025/2026

Data di laurea: xx/07/2026





UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA

## SOMMARIO

Il seguente progetto nasce dall'idea di applicare i principi del Digital Signal Processing (DSP) all'ambito musicale, realizzando un pedale multieffetto basato su microcontrollore ispirato ad altri multieffetto digitali presenti sul mercato, come l'Empress Zoia ed il Boss zoom g1x. L'attività di sviluppo ha affrontato varie tematiche di sviluppo tra cui la modellazione di sistemi non lineari e la modulazione in ampiezza e frequenza, come anche limiti legati alla gestione di risorse computazionali e di memoria.

L'obiettivo è fornire un riscontro pratico a nozioni teoriche di DSP, valutandone il comportamento in un sistema reale caratterizzato da vincoli stringenti di latenza e prestazioni.

Repository github:

<https://github.com/Alemagpie/Teensy-based-Multieffect-Pedal>



## INDICE

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>Elenco delle figure</b>                      | <b>iv</b> |
| <b>1 Introduzione</b>                           | <b>1</b>  |
| 1.1 Obiettivi del progetto . . . . .            | 1         |
| 1.2 Hardware . . . . .                          | 1         |
| 1.3 Teoria DSP di base . . . . .                | 3         |
| 1.4 Guida alla lettura dei grafici . . . . .    | 4         |
| <b>2 Microcontrollore</b>                       | <b>7</b>  |
| 2.1 Scelta del microcontrollore . . . . .       | 7         |
| 2.2 Caratteristiche tecniche . . . . .          | 7         |
| <b>3 Circuiti elettronici</b>                   | <b>9</b>  |
| 3.1 Buffer di input . . . . .                   | 9         |
| 3.2 Buffer di output . . . . .                  | 10        |
| 3.3 Circuito di alimentazione . . . . .         | 11        |
| 3.4 Realizzazione su schede . . . . .           | 12        |
| <b>4 Software</b>                               | <b>13</b> |
| 4.1 Architettura e pipeline audio . . . . .     | 13        |
| 4.2 Effetto distorsione . . . . .               | 14        |
| 4.3 Effetto tremolo . . . . .                   | 19        |
| 4.4 Effetto bitcrusher . . . . .                | 24        |
| 4.5 Effetto vibrato ed effetto chorus . . . . . | 28        |
| 4.5.1 Effetto vibrato . . . . .                 | 28        |
| 4.5.2 Effetto chorus . . . . .                  | 31        |
| 4.6 Caricamento e salvataggio . . . . .         | 34        |
| 4.7 Interfaccia grafica . . . . .               | 35        |
| <b>Conclusioni</b>                              | <b>37</b> |



## ELENCO DELLE FIGURE

|      |                                                                          |    |
|------|--------------------------------------------------------------------------|----|
| 1.1  | Foto frontale del pedale . . . . .                                       | 2  |
| 1.2  | Esempio di campionamento di segnali a diverse frequenze . . . . .        | 3  |
| 2.1  | Benchmark di vari microcontrollori eseguito da Paul Stoffregen . . . . . | 7  |
| 3.1  | Schema circuitale del buffer di input . . . . .                          | 9  |
| 3.2  | Schema circuitale del buffer di output . . . . .                         | 10 |
| 3.3  | Schema circuitale del circuito di alimentazione . . . . .                | 11 |
| 3.4  | Visualizzazione progettuale . . . . .                                    | 12 |
| 3.5  | Render 3D della scheda . . . . .                                         | 12 |
| 4.1  | Schema logico dell'architettura software . . . . .                       | 13 |
| 4.2  | Funzione tanh . . . . .                                                  | 15 |
| 4.3  | Oscillogramma di una funzione seno a 400 Hz . . . . .                    | 15 |
| 4.4  | Funzione con gain al minimo ed al massimo . . . . .                      | 16 |
| 4.5  | Oscillogramma dell'output con gain a 12 . . . . .                        | 16 |
| 4.6  | Funzione di elaborazione con bias al minimo ed al massimo . . . . .      | 17 |
| 4.7  | Oscillogramma dell'output con bias a 0 . . . . .                         | 17 |
| 4.8  | Oscillogramma dell'output con bias a 0.5 . . . . .                       | 17 |
| 4.9  | Analisi in frequenza dell'output con bias a 0 . . . . .                  | 17 |
| 4.10 | Analisi in frequenza dell'output con bias a 0.5 . . . . .                | 17 |



|      |                                                                             |    |
|------|-----------------------------------------------------------------------------|----|
| 4.11 | Oscillogramma dell'output con filtro totalmente chiuso . . . . .            | 18 |
| 4.12 | Oscillogramma dell'output con filtro totalmente aperto . . . . .            | 18 |
| 4.13 | Spettrogramma dell'output con filtro totalmente chiuso . . . . .            | 18 |
| 4.14 | Spettrogramma dell'output con filtro totalmente aperto . . . . .            | 18 |
| 4.15 | Onda sinusoidale modulata da un'onda triangolare .                          | 19 |
| 4.16 | 5 tipi d'onda implementati . . . . .                                        | 20 |
| 4.17 | Differenza tra onda bipolare ed unipolare . . . . .                         | 20 |
| 4.18 | Segnale modulato da un'onda a rampa a frequenze                             | 22 |
| 4.19 | diverse . . . . .                                                           |    |
| 4.20 | Segnale modulato da un'onda a dente di sega . . . . .                       | 22 |
| 4.21 | Segnale modulato da un'onda sinusoidale . . . . .                           | 22 |
| 4.22 | Segnale modulato da un'onda sinusoidale con profondità a metà . . . . .     | 23 |
| 4.23 | Segnale modulato da un'onda sinusoidale con profondità al massimo . . . . . | 23 |
| 4.24 | Esempio di quantizzazione a varie risoluzioni . . . . .                     | 24 |
| 4.25 | Oscillogramma di due onde a 200 Hz e 500 Hz sovrapposte . . . . .           | 25 |
| 4.26 | Oscillogramma con risoluzione di bit dimezzata . . .                        | 26 |



|      |                                                                     |    |
|------|---------------------------------------------------------------------|----|
| 4.27 | Oscillogramma con frequenza di campionamento a 2.76 kHz . . . . .   | 26 |
| 4.28 | Oscillogramma con frequenza di campionamento a 1.38 kHz . . . . .   | 26 |
| 4.29 | Analisi in frequenza con frequenza di sampling a 2.76 kHz . . . . . | 27 |
| 4.30 | Analisi in frequenza con frequenza di sampling a 1.38 kHz . . . . . | 27 |
| 4.31 | Esempio di modulazione in frequenza . . . . .                       | 28 |
| 4.32 | Schema logico del funzionamento di un vibrato digitale . . . . .    | 29 |
| 4.33 | Spettrogrammi di 400 Hz modulati con due fre-                       | 30 |
| 4.34 | quenze diverse . . . . .                                            |    |
| 4.35 | Spettrogrammi di 400 Hz modulati con due diverse                    | 30 |
| 4.36 | profondità . . . . .                                                |    |
| 4.37 | Schema logico del funzionamento di un chorus digitale . . . . .     | 31 |
| 4.38 | Segnale a 400 Hz modulato con 4 voci . . . . .                      | 33 |
| 4.39 | Interfaccia grafica . . . . .                                       | 36 |





# Capitolo 1

## INTRODUZIONE

### 1.1 - Obiettivi

Lo scopo di questo progetto è di realizzare una codebase performante in grado di supportare catene modulari di effetti costruiti a partire da blocchi elementari quali filtri passa alte/passa basso, quantizzatori ed oscillatori a bassa frequenza (LFO) appositamente realizzati.

Gli effetti implementati nella realizzazione della tesi sono i seguenti:

- distorsione (emulazione di sistema non lineare)
- tremolo (modulazione in ampiezza)
- bitcrusher (quantizzazione e downsampling)
- vibrato (modulazione in frequenza)
- chorus (modulazione a più voci)

### 1.2 - Hardware

All'interno del pedale si possono trovare un circuito stampato comprendente di buffer di input/output, circuito di alimentazione ed un microcontrollore Teensy 4.1 accoppiato con una scheda audio Audio Shield. All'esterno il pedale è composto da tre sezioni principali: il display OLED, i potenziometri dei parametri ed i pulsanti di selezione e attivazione degli effetti.

L'involucro esterno è una scatola di derivazione elettrica in plastica ABS: questo materiale si presta bene a questo progetto in quanto è resistente a sollecitazioni meccaniche e facile da lavorare. La superficie interna è stata rivestita da nastro adesivo in alluminio, per isolare i circuiti interni e proteggerli da interferenze elettromagnetiche. Tutti gli elementi esterni (jack audio, pulsanti, potenziometri, ecc.) sono collegati alla scheda stampata tramite connettori JST, per favorire modularità ed intercambiabilità.

La scelta del display “SSD1309 OLED 1.54” ” è stata dettata sia dalla capacità dello stesso di comunicare con il microcontrollore tramite protocollo I2C (che richiede solo 4 pin invece dei 7 necessari con protocollo SPI), che dalle dimensioni fisiche, così da poter mostrare all’utente sufficienti informazioni senza eccessivo ingombro.

La lettura dei parametri avviene tramite 4 potenziometri, ed è abilitata da un pulsante affiancato da un diodo LED che ne indica lo stato (entrambi sotto lo schermo). I segnali dei potenziometri vengono convertiti in valori digitali (ogni 100ms per ragioni di performance e stabilità), per poi essere utilizzati nel software.

Tramite i tre pulsanti momentanei SPST, l’utente può accedere ad una serie di funzioni in base a quanto tempo il pulsante è schiacciato: premere a lungo il pulsante centrale alterna lo stato tra “esecuzione” e “modifica”. In modalità esecuzione i due pulsanti laterali permettono di scorrere la catena di effetti mentre una breve pressione di quello centrale attiva e disattiva l’effetto selezionato. Dall’altra parte in modalità modifica i pulsanti laterali permettono di scegliere l’effetto da inserire, ed una pressione breve lo conferma, riportando il sistema in modalità esecuzione.



*Fig. 1.1 - Foto frontale del pedale*

### 1.3 - Teoria DSP di base

Nell'elaborazione audio digitale il passaggio dal dominio analogico a quello digitale avviene tramite il campionamento, ovvero la rilevazione di valori del segnale ad intervalli regolari, effettuato attraverso convertitori Analogico-Digitale (ADC).

Il campionamento consente di acquisire un segnale analogico, convertirlo in forma digitale e successivamente ricostruirlo in dominio analogico senza perdita di qualità, purché il suo contenuto spettrale non superi una soglia detta frequenza di Nyquist, indicata con  $f_s$ , pari alla metà della frequenza di campionamento.

Quando il segnale contiene componenti superiori a tale limite, si verifica il fenomeno dell' "aliasing", ossia la comparsa di frequenze spurie (alias) dovute alla rappresentazione ambigua delle frequenze troppo elevate. Questo avviene poiché per descrivere correttamente un'onda, sono necessari più di due campioni per periodo, perciò lo standard audio prevede una frequenza di campionamento minima di 44.1 kHz, valore leggermente superiore al doppio del limite dell'udibile umano (circa 20 kHz).

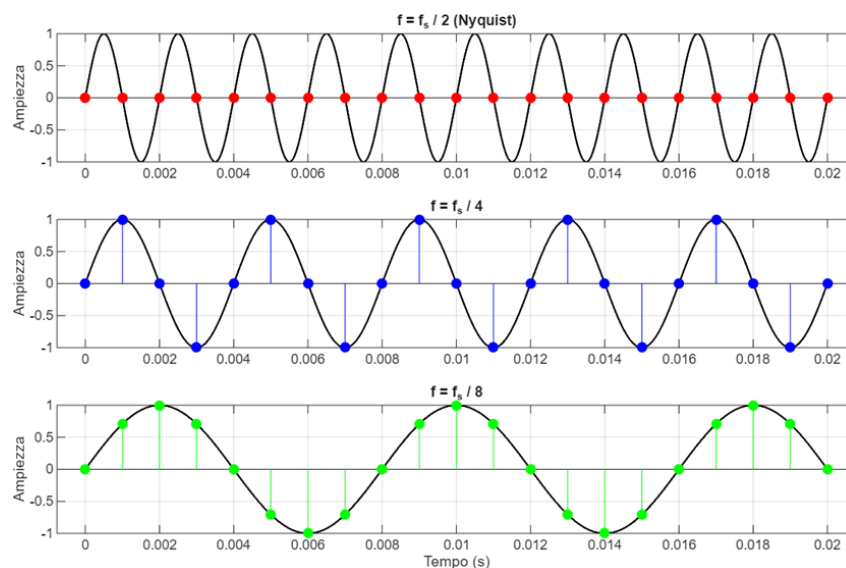


Fig. 1.2 - Esempio di campionamento di segnali a diverse frequenze



Prima dell'uscita, il segnale analogico ricostruito attraversa un filtro passa basso, per eliminare eventuale rumore digitale dovuto alla risposta elettrica delle capacità e induttanze parassite allo switching elettronico ad alta frequenza. In seguito alla conversione in digitale del segnale, si può scegliere di rappresentarlo in vari modi: si può usare la rappresentazione in floating point, che richiede meno accortezze ed è di più facile comprensione, oppure la rappresentazione con interi (a diverse profondità di bit), che richiede più attenzione ma è generalmente più performante.

Il problema principale della rappresentazione con interi è l'overflow, dato che con  $2^N$  bit si possono rappresentare  $2^{(N-1)}$  livelli di quantizzazione negativi e  $2^{(N-1)} - 1$  livelli positivi.

In questo progetto la frequenza di campionamento è 44.1kHz e la rappresentazione dei sample è a 16 bit con segno, indicata come Q15.

## 1.4 - Guida alla lettura dei grafici

**Oscillogrammi:** gli oscillogrammi visualizzano l'andamento del segnale nel tempo rappresentandone l'ampiezza. Più l'onda si estende in altezza, maggiore è la sua ampiezza; più volte sono rappresentate le oscillazioni, maggiore è la frequenza.

**Spettrogrammi:** gli spettrogrammi rappresentano il contenuto armonico del segnale nel tempo. Ogni frequenza dello spettro sonoro è rappresentata da una certa altezza sulla scala verticale e da un colore, più è chiaro il colore e maggiore è l'intensità sonora.

**Analisi in frequenza:** le analisi in frequenza sono immagini che mostrano l'intensità dello spettro sonoro in un dato istante. Ogni picco rappresenta una frequenza; in ambito musicale il picco più a sinistra è generalmente il più esteso e rappresenta la “fondamentale” del suono, ovvero la frequenza primaria. I picchi secondari posti a distanza regolare sono le armoniche, ovvero le frequenze multiple della fondamentale.



Gli oscillogrammi e le analisi in frequenza sono state ottenute grazie al software “LSaO Visualizer”, mentre gli spettrogrammi sono stati realizzati con il programma “Sonic Visualizer”.



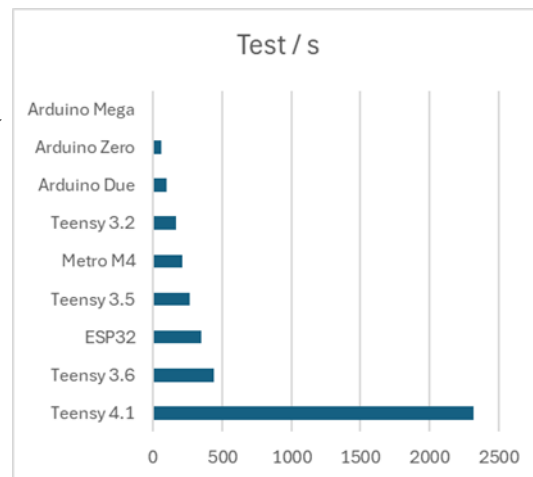


## Capitolo 2

### MICROCONTROLLORE

#### 2.1 - Scelta del microcontrollore

Data la natura del progetto, la scelta del microcontrollore è stata orientata alle schede più performanti. Il Teensy 4.1 risulta in cima alle classifiche di benchmark delle schede più comuni in circolazione, ma è stato necessario incorporare la scheda di espansione “Audio Shield”, ottimizzata per istruzioni su segnali audio e dotata di una libreria specifica a riguardo.



*Fig. 2.1 - Benchmark di vari microcontrollori (asse verticale) eseguito da Paul Stoffregen*

#### 2.2 - Caratteristiche tecniche

Il Teensy 4.1 è dotato di un processore ARM Cortex M7 con frequenza di clock di 600 MHz, 1MB di RAM (di cui 512KB sono tightly coupled e l'altra metà è ottimizzata per DMA), espandibile con due chip esterni da 8MB l'uno, ed un totale di 8MB di memoria flash per caricare programmi. Il Cortex M7 è dotato di software di predizione di branch ed un set di istruzioni esteso per il DSP, il che lo rende ancora più adatto all'esecuzione di loop di elaborazione audio.

Dall'altra parte la scheda “Audio Shield”, usata in modalità slave, comunica con la scheda Teensy tramite protocollo I2C e permette di gestire segnali audio stereo contemporaneamente a 44.1kHz, sia in input che in output. La scheda è dotata di chip codec SGTL5000, che controlla i segnali delle ADC e DAC<sup>1</sup>.

<sup>1</sup> ADC = Analog to Digital Converter, DAC = Digital to Analog Converter



## Capitolo 3

### CIRCUITI ELETTRONICI

#### 3.1 - Buffer di input

Il buffer di input ha la funzione di trasmettere il segnale analogico in entrata proveniente dai pickup<sup>2</sup> dello strumento all' ADC dell' Audio Shield, adattando l'impedenza e preservandone il contenuto spettrale.

I pickup generalmente hanno impedenza di uscita elevata (da 10kΩ a più di 100kΩ) e sono molto sensibili al carico ed alla capacità del circuito successivo, mentre l'ADC richiede un segnale centrato su 1.65V (range complessivo da 0V a 3.3V) con bassa impedenza. Un'errata corrispondenza di impedenza porta ad attenuazione delle frequenze alte, perdita di dinamica e variazione di timbro in base al potenziometro del volume dello strumento.

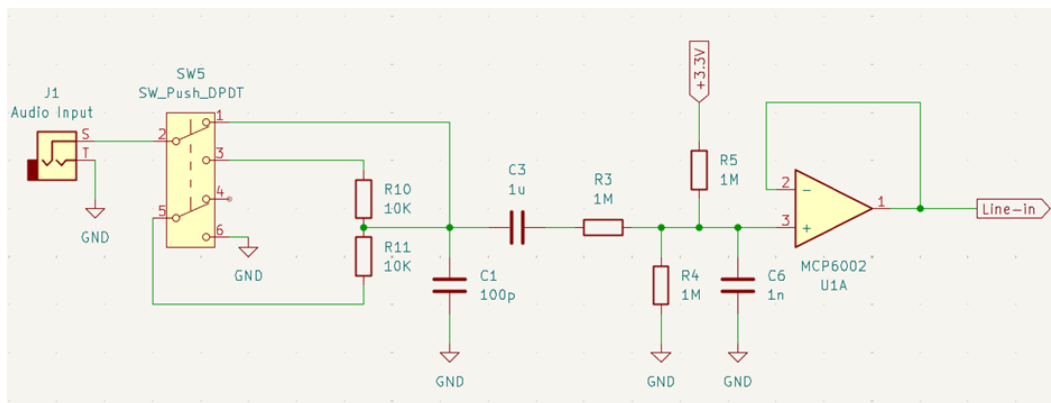


Fig. 3.1 - Schema circuitale del buffer di input

L'op-amp (MCP6002) è utilizzato in modalità non-invertente a guadagno unitario, garantendo quindi bassa impedenza in uscita. L'interruttore DPDT all'inizio dell'input dell'op-amp permette di attenuare il segnale attraverso il divisore di voltaggio (R10, R11), evitando che nel caso in cui sia collegato un pickup attivo<sup>3</sup>, il segnale sia troppo ampio e venga distorto. Nell'altro stato dello switch, il segnale rimane invariato e la messa a terra del divisore di voltaggio è scollegata, per evitare carichi eccessivi sul pickup e modifiche non volute al segnale.

Il segnale in entrata riceve un bias di 1.65V tramite il divisore di voltaggio da 3.3V (R4, R5) ed è filtrato attraverso un filtro passa alte (R3 + R4 // R5, C3) con frequenza di taglio di 0.1Hz, usato per bloccare eventuali correnti continue.

<sup>2</sup> pickup è un termine usato per indicare il dispositivo che negli strumenti a corde metalliche traduce la vibrazione delle stesse in segnale elettrico, è composto da un induttore di migliaia di spire avvolte attorno ad un magnete

<sup>3</sup> a differenza dei pickup passivi, che sono semplici induttori, i pickup attivi sono dotati di un circuito aggiuntivo di equalizzazione e buffering, quindi il segnale ha ampiezza massima maggiore e impedenza minore

### 3.2 - Buffer di output

Il buffer di output serve ad adattare il segnale in uscita dal DAC per renderlo adeguato ad apparecchiature esterne, come amplificatori o pedali d'effetto. Il DAC restituisce un segnale centrato su 1.65V che va traslato su 0V, ed in aggiunta il buffer garantisce impedenza d'uscita bassa per poter pilotare carichi variabili (dai 10kΩ ai 100kΩ).

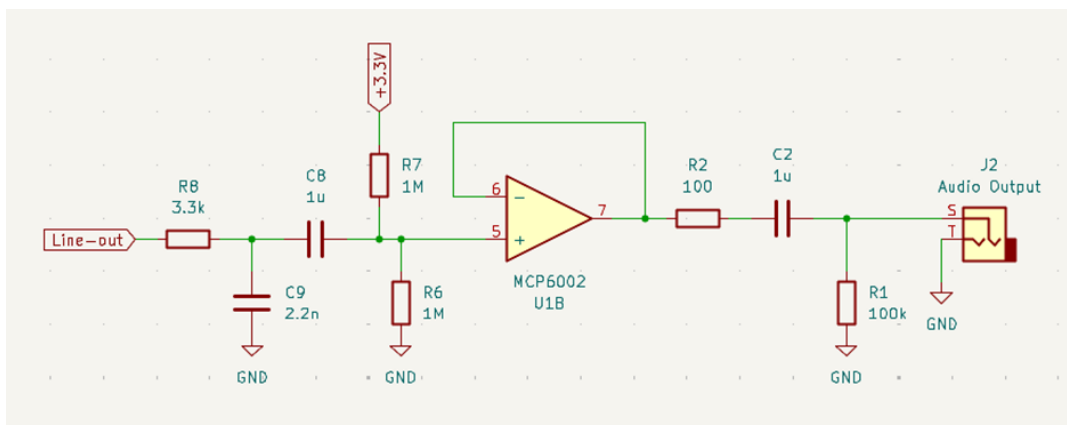


Fig.3.2 - Schema circuitale del buffer di output

Analogamente al buffer di ingresso, il buffer di uscita si avvale di un op-amp in modalità non-invertente a guadagno unitario. Il divisore di voltaggio (R6, R7) fornisce il bias per centrare il segnale su 0V, il filtro passa basso (R8, C9) taglia le frequenze superiori a 21.9kHz ed i due filtri passa alte (R6 // R7, C8 e R1, C2) tagliano rispettivamente le frequenze superiori a 0.32Hz e 1.59Hz, bloccando correnti continue.

### 3.3 - Circuito di alimentazione

Il circuito di alimentazione è formato da un diodo Schottky per la protezione da polarità inversa e due regolatori lineari di tensione. I regolatori di tensione sono rispettivamente da 3.3V (usato per alimentare l'op-amp ed i circuiti dei parametri di input) e 5V (usato per alimentare il microcontrollore). Entrambi i regolatori dispongono in uscita di condensatori di disaccoppiamento per stabilizzare eventuali fluttuazioni di alimentazione. La tensione in uscita dal diodo (VCC, si aggira tra gli 8.55V e 8.85V) è usata per alimentare lo schermo OLED.

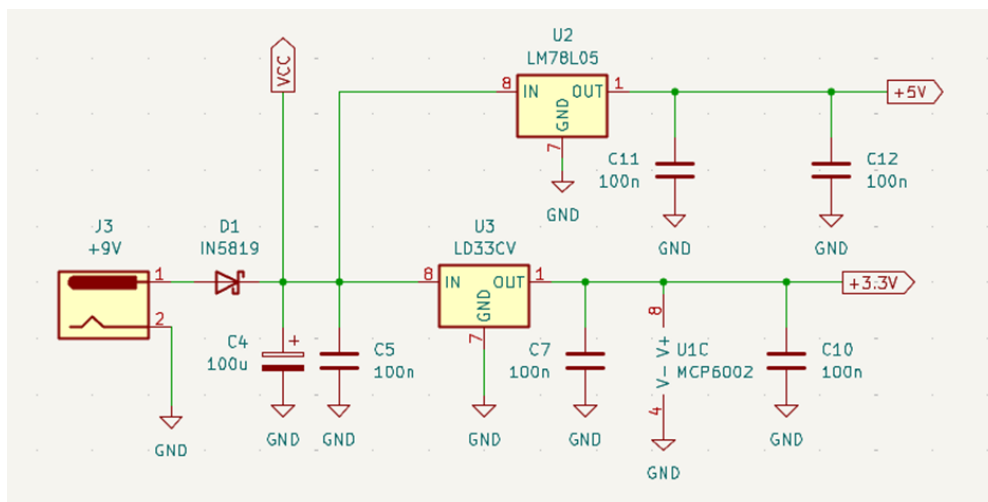


Fig.3.3 - Schema circuitale del circuito di alimentazione

### 3.4 - Realizzazione su scheda

I circuiti mostrati sono stati progettati su KiCad ed assemblati dalla ditta JLCPCB su una scheda a 2 layer. I componenti più piccoli, ovvero resistenze e condensatori, sono stati applicati superficialmente in fabbrica (componenti SMD), mentre quelli con montaggio a foro passante (connettori, diodi, op-amp, switch e regolatori di tensione) sono stati saldati manualmente. Per garantire meno interferenza possibile tra la sezione digitale e quella analogica del circuito, il layer inferiore della scheda è stato adibito ad essere interamente di messa a terra.

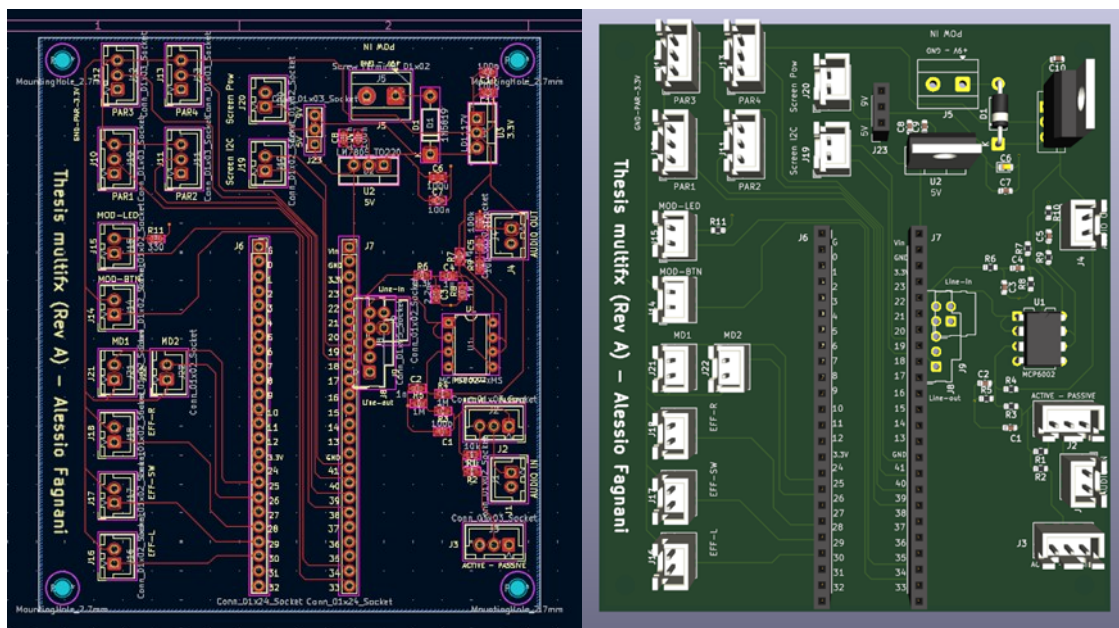


Fig. 3.4, 3.5 - Visualizzazione progettuale (sinistra) e render 3D (destra) della scheda



## Capitolo 4

### SOFTWARE

#### 4.1 - Architettura e pipeline audio

Il sistema funziona come una macchina a stati, gli input dell'utente rappresentano gli eventi che ne innescano le transizioni. Gli stati possibili sono tre: "PLAY", in cui l'utente ha accesso solo alla catena di effetti e può solo abilitarli e disabilitarli, "MODIFY", che estende il precedente e permette la modifica dei valori dei parametri ed "EDIT", in cui l'utente può scegliere da una lista di effetti disponibili da introdurre nella catena.

La libreria audio usata dal microcontrollore mette a disposizione una pipeline costituita da oggetti di tipo "AudioStream", capaci di ricevere input e trasmettere output, collegati tra di loro da oggetti di tipo "AudioConnection". Per dare un'interfaccia comune agli effetti implementati ed evitare di modificare la classe AudioStream (da cui dipendono numerose altre classi di vario genere), è stata creata la classe astratta pura EffectAdapter. Le funzionalità delle due classi sono indipendenti e l'implementazione di una non influisce sull'altra (AudioStream si occupa della trasmissione audio nel core ed EffectAdapter si occupa di far comunicare l'effetto col resto del sistema). In particolare, EffectAdapter gestisce i parametri (che hanno diversi range e funzioni in base all'effetto) e l'accensione/spengimento dell'effetto. Una terza classe "ScreenManager" gestisce l'interfaccia grafica dello schermo OLED.

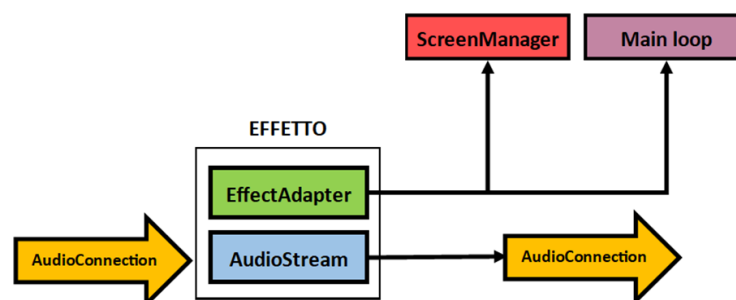


Fig. 4.1 - Schema logico dell'architettura software



La programmazione è avvenuta prevalentemente in C++, con brevi usi di Java per la generazione dei valori delle LookUp Table.

La valutazione delle prestazioni di ciascun effetto è avvenuta misurando l'utilizzo di CPU e RAM tramite le funzioni “*AudioProcessorUsage()*” e “*AudioMemoryUsage()*”, messe a disposizione dalla libreria audio.

L'intero software (effetti, macchina a stati, gestione degli input, gestione del display, gestione del salvataggio) occupa circa 117 KB di memoria Flash, pari a circa l'1.42% della capacità totale.

## 4.2 - Effetto distorsione

In ambito audio la distorsione è un effetto che altera il segnale originale modificando la forma d'onda ed aggiungendo armoniche. Il principio che c'è dietro a questo effetto è il “clipping”, ovvero il taglio netto della porzione d'onda che supera una certa ampiezza.

Il modo in cui la forma d'onda è manipolata influisce molto sul tipo di armoniche prodotte: se la funzione di elaborazione è simmetrica tra range positivo e negativo verranno generate armoniche uguali a multipli dispari della fondamentale, che donano un timbro percepito come brillante e tagliente; se invece la funzione è asimmetrica (i valori negativi sono elaborati diversamente da quelli positivi) le armoniche create saranno multipli pari, che danno calore e saturazione al suono.

Nell'ambito analogico la distorsione è stata storicamente ottenuta in svariati modi: i primi amplificatori utilizzavano valvole termoioniche, poi rimpiazzate dai più compatti transistor ed op-amp. Nel caso di questo progetto, l'effetto implementato mira ad emulare il comportamento tipico non lineare di un circuito con clipping a transistor / diodo.

La funzione scelta per simulare il circuito è la tangente esponenziale “ $\tanh(x)$ ”, caratterizzata da un'immagine limitata ed una curva di saturazione molto graduale (soft clip). La curva è simmetrica, quindi di base verranno prodotte armoniche dispari.

La funzione finale è la seguente:  $y(x) = p_3 * f_{p4}(\tanh(p_1 * x + p_2))$ , dove  $p_i$  è l'i-esimo parametro ed  $f_{p4}$  è la funzione caratteristica del filtro regolato dal parametro n. 4. Il segnale di test sarà un segnale sinusoidale a 400 Hz.

```
void DistortionEffect::processSignal(int16_t &value) {  
    //Scale by the overall volume  
    value = signed_saturate_rshift(  
        //Feed value into tanh function  
        volume * Utility::fastTanh(  
            //Apply gain and bias, then clip  
            saturate16(  
                (int32_t)gain * value + bias  
            )  
        ),  
        16, 15);  
}
```

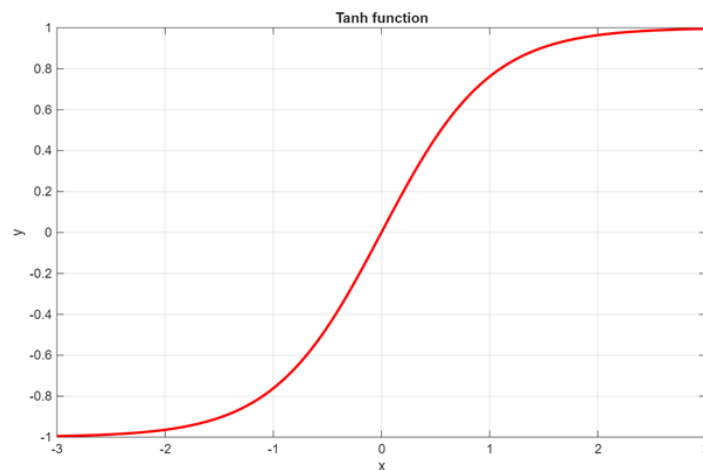


Fig. 4.2 - Funzione tanh

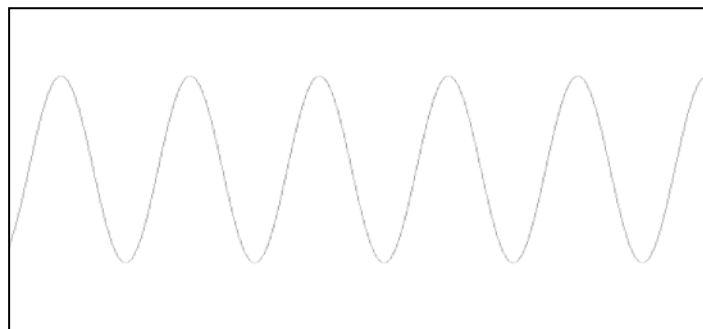
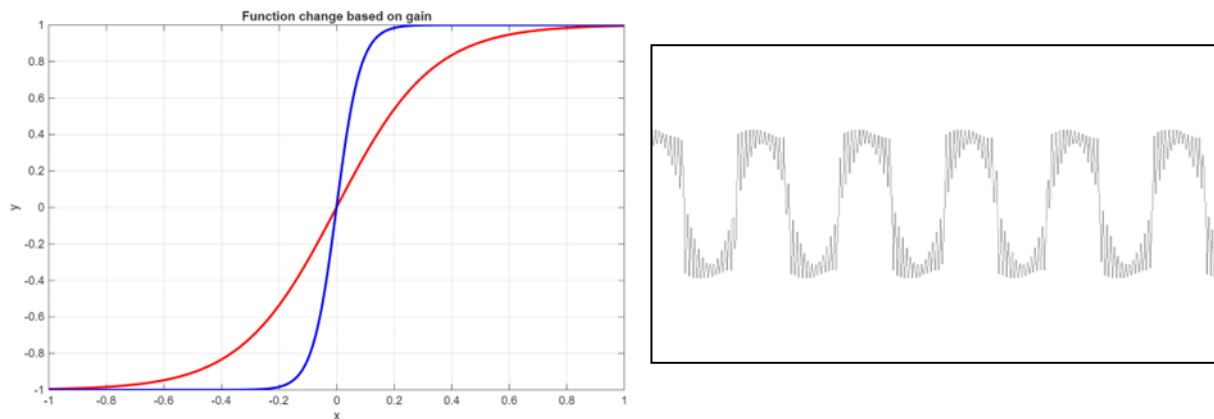


Fig. 4.3 - Oscillogramma di una funzione seno a 400 Hz

Nella classe “distortion\_effect.h” tre dei quattro parametri disponibili sono stati mappati a variabili che vanno a cambiare la forma della curva, mentre il quarto parametro controlla un filtro passa basso:

**Parametro 1** - controlla il **guadagno** della funzione. Aumentando il valore del parametro la funzione di elaborazione viene “appiattita” sulle x, rendendola più ripida e creando una compressione del suono maggiore. Il segnale risulta espanso verticalmente e tagliato dove supera il valore massimo di ampiezza. Il gain va da 3 a 12.



*Fig. 4.4, 4.5 - Funzione con gain al minimo (in rosso) ed al massimo (in blu); Oscillogramma dell’output con gain a 12 (a destra)*

**Parametro 2** - varia il **bias** del segnale di input, in un circuito analogico equivale ad aggiungere tensione continua al segnale. Aumentando il bias la funzione di elaborazione smette di essere simmetrica e l’effetto introduce armoniche pari. Il bias va 0 a 16 380 che, in Q15 normalizzato, corrisponde al range  $[0, \sim 0.5]$ .

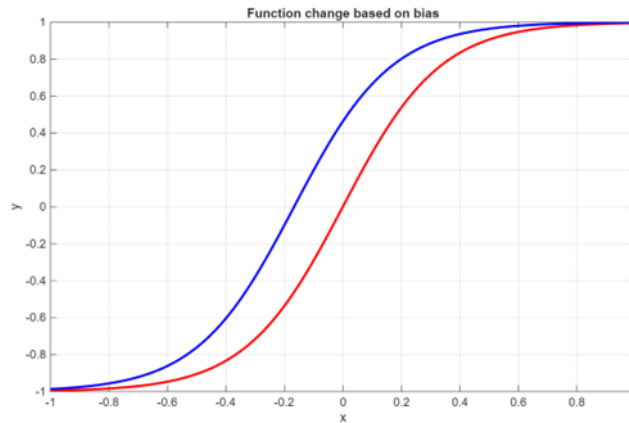


Fig. 4.6 - Funzione di elaborazione con bias al minimo (rosso) ed al massimo (blu)

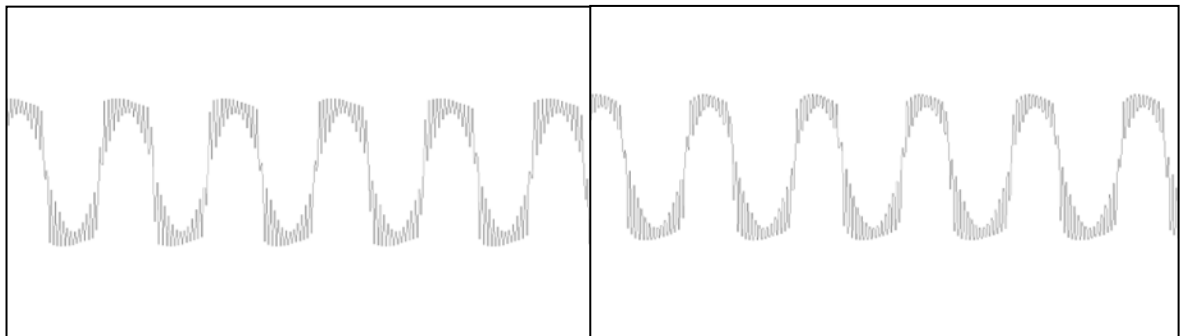


Fig. 4.7, 4.8 - Oscillogramma dell'output con bias rispettivamente a 0 ed a 0.5 (nella seconda immagine si può notare come l'onda sia distorta in maniera asimmetrica tra le porzioni positive e negative)

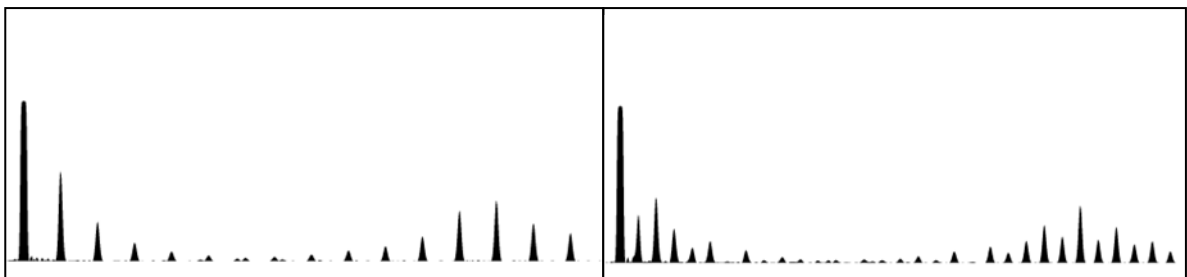


Fig. 4.9, 4.10 - Analisi in frequenza dell'output con bias rispettivamente a 0 ed a 0.5 (nella prima immagine sono presenti solo armoniche dispari, mentre nella seconda sono presenti anche armoniche pari)

**Parametro 3** - controlla il **volume** del segnale di output, quindi schiaccia la funzione sulle y: valori maggiori del parametro equivalgono a segnali più intensi in uscita. Il volume va da 0 a 32767, che corrisponde a  $[0, \sim 1]$ .

**Parametro 4** - modifica un filtro passa basso ad 1 polo<sup>4</sup> posto in uscita alla funzione: più aumenta il valore del parametro, maggiore sarà la frequenza di taglio del filtro, quindi minore sarà la riduzione del volume delle frequenze alte. La frequenza di taglio del filtro va da 2500Hz a 8000Hz.

<sup>4</sup> I filtri hanno uno o più poli. Il numero di poli determina la ripidità della curva di attenuazione, la stabilità del sistema e la frequenza caratteristica di risonanza del filtro

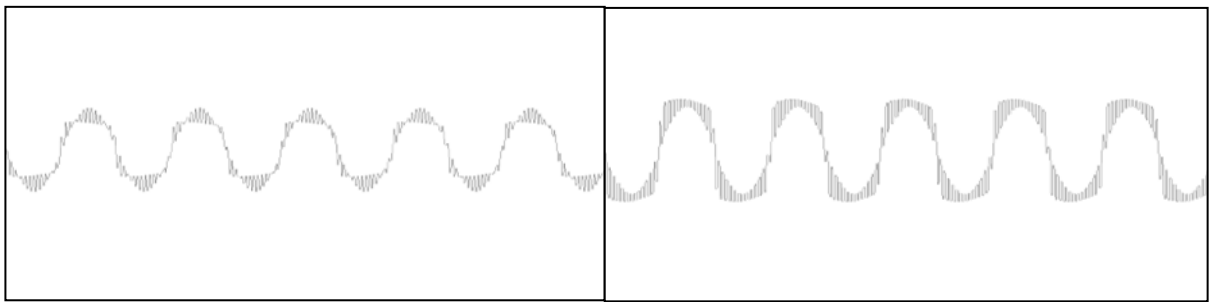


Fig. 4.11, 4.12 - Oscillogramma dell'output con filtro totalmente chiuso (a sinistra) e totalmente aperto (a destra)

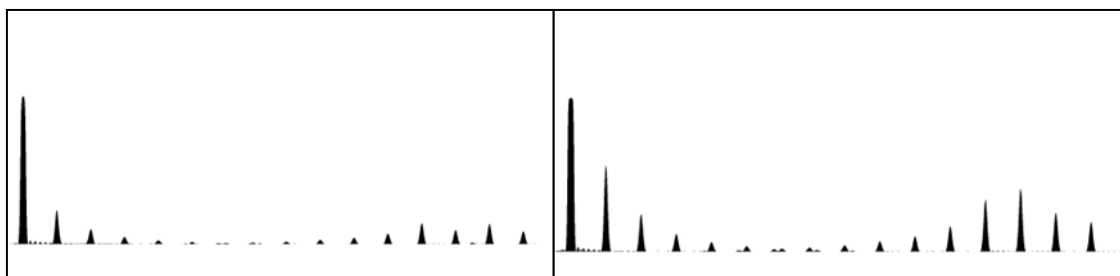


Fig. 4.13, 4.14 - Spettrogramma dell'output con filtro totalmente chiuso (a sinistra) e totalmente aperto (a destra), nella prima immagine si possono notare le frequenze alte attenuate

Dato che la funzione tangente esponenziale è eccessivamente complessa da calcolare decine di migliaia di volte al secondo, è stata implementata una “LookUpTable” (LUT) che permette di approssimarne i valori tramite una tabella pregenerata ed una funzione di interpolazione lineare. Per ovvie ragioni di performance la tabella dei valori è memorizzata nella memoria Flash anziché nella RAM.

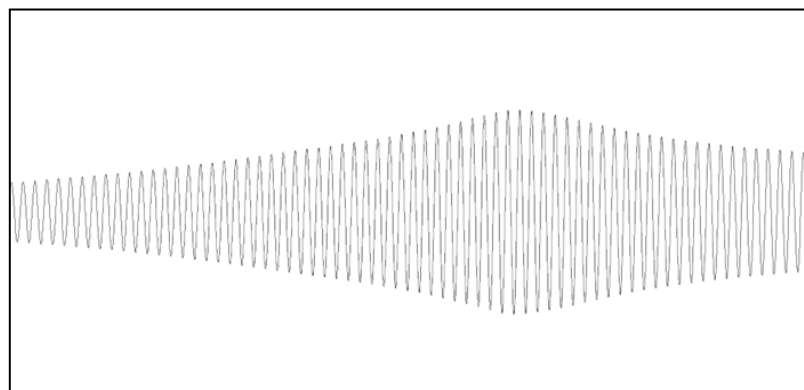
L’impiego a regime della CPU si aggira sul 2.17%, mentre la RAM utilizzata è pari a 58.7 KB, cioè il 5.73%.

### 4.3 - Effetto tremolo

L’effetto “tremolo” corrisponde alla modulazione in ampiezza del segnale in entrata in base ad un’onda generata da un oscillatore a bassa frequenza (LFO - Low Frequency Oscillator).

Il tremolo è esistito fin dalla metà dagli anni ‘40: facendo ruotare degli altoparlanti attorno ad un asse<sup>5</sup> si otteneva una variazione ciclica dell’intensità sonora (e della frequenza, a causa dell’effetto doppler), regolabile tramite la velocità di rotazione dell’apparato. Nel dominio digitale questo effetto consiste nella moltiplicazione tra i campioni del segnale di input e dell’onda modulatrice.

<sup>5</sup> un esempio sono i famosi “Leslie rotary speakers”, creati da Donald Leslie nel 1945



*Fig. 4.15 - Onda sinusoidale modulata da un’onda triangolare*

Gli LFO generalmente hanno frequenza compresa tra 0.1 Hz e 20 Hz e generano 5 tipi di onda: sinusoidale, quadra, triangolare, a dente di sega e a rampa.

Le onde che oscillano attorno allo 0 sono dette onde bipolari, mentre quelle che, mantenendo la stessa forma oscillano solo su valori positivi, sono dette onde unipolari. Per realizzare questo effetto è necessario trasformare le onde prodotte dall' LFO in onde unipolari, dato che il volume (normalizzato) di un segnale non può essere negativo e va da 0 ad 1.

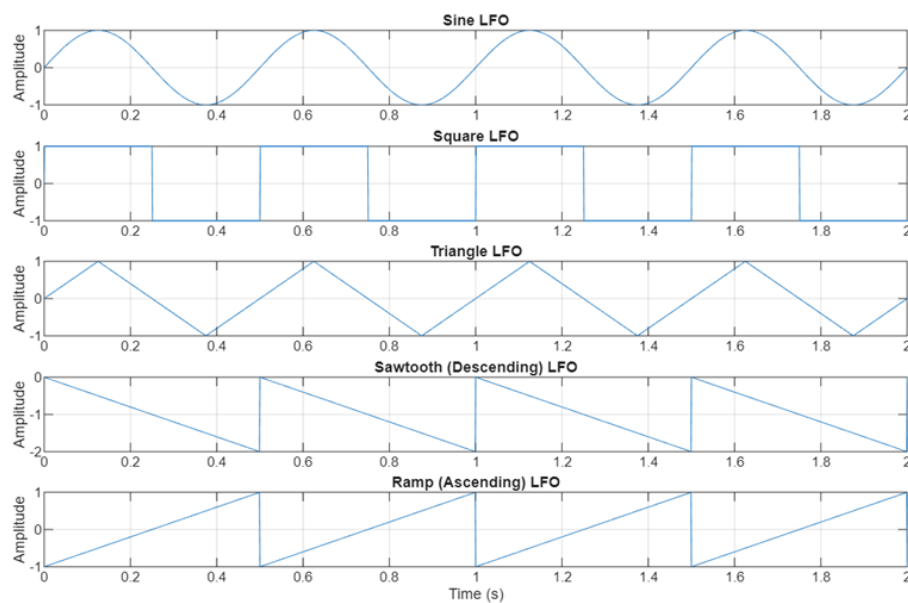


Fig. 4.16 - 5 tipi d'onda implementati

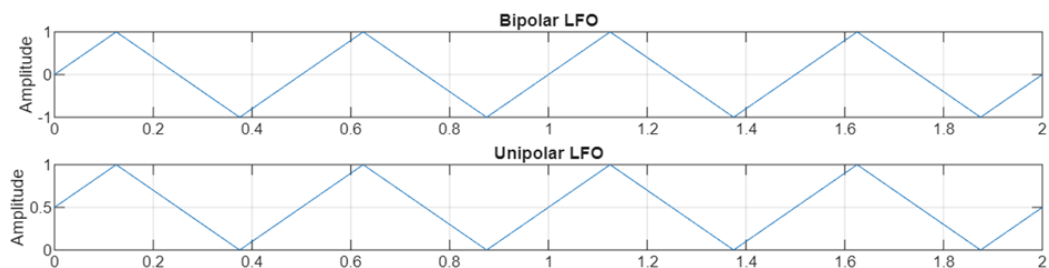


Fig. 4.17 - Differenza tra onda bipolare ed unipolare



La classe “LFO.h” è stata implementata per essere una “helper class” dell’effetto e, per motivi di efficienza, al posto di generare campioni continui riempie un buffer solo su richiesta. Il generatore d’onda è stato implementato tramite accumulatore di fase, ovvero un contatore incrementato periodicamente di un valore determinabile in base alla frequenza. Il contatore rappresenta per l’appunto la fase accumulata, che determina il valore d’uscita dell’onda. Il segnale sinusoidale è generato mediante una Look-Up Table con interpolazione lineare, mentre le restanti forme d’onda sono ottenute tramite espressioni matematiche.

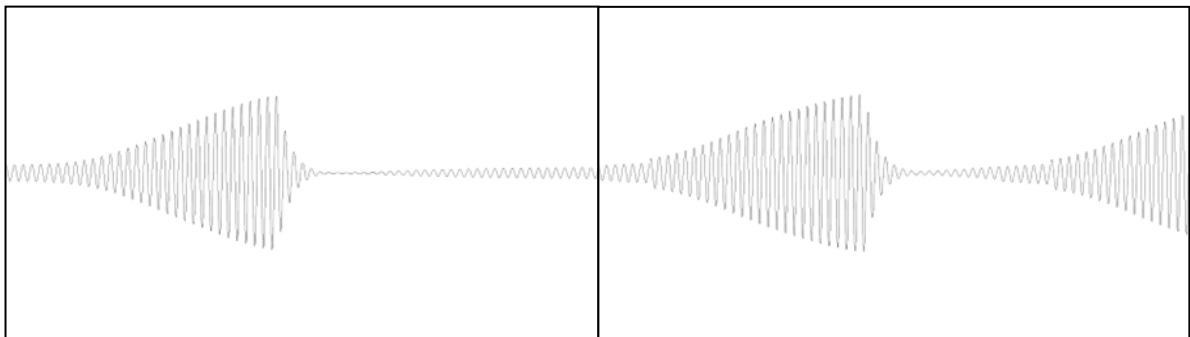
Prima di impiegare il segnale dell’LFO nella modulazione, è stato necessario filtrarlo attraverso un filtro passa basso, con frequenza di taglio fissata a 100 Hz, al fine di attenuare le componenti ad alta frequenza generate dalle discontinuità delle forme d’onda non sinusoidali (come onda quadra, a sega e a rampa). Tali discontinuità, corrispondenti a variazioni impulsive del segnale, si sarebbero manifestate nel segnale modulato in artefatti transitori di ampiezza elevata.

```
void doTremolo() {
    for(int i = 0; i < AUDIO_BLOCK_SAMPLES; i++) {
        //Filter lfo and scale it by depth
        lp_m.process(lfoSample);
        lfoSample = signed_saturate_rshift(
            depth * lfoSample,
            16,
            15
        );
        g_m.setGain(lfoSample);
        g_m.process(sample);

        *inputSamplePtr = sample;
        inputSamplePtr++;
        lfoSamplePtr++;
    }
}
```

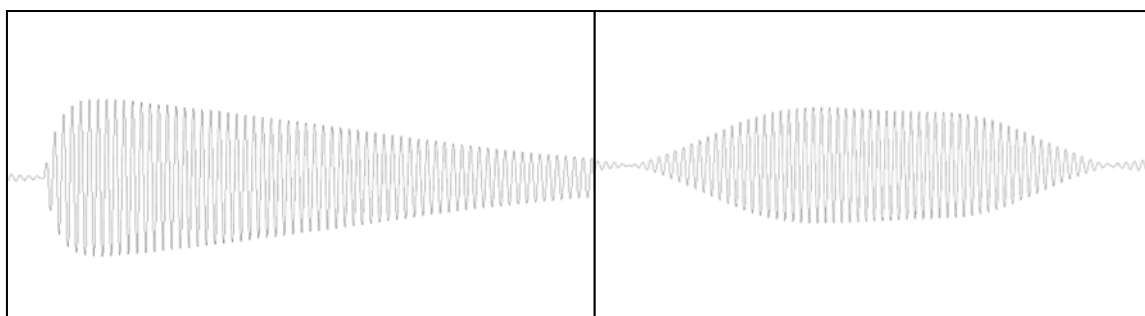
La classe corrispondente all'implementazione di questo effetto è "tremolo\_effect.h". Tre dei quattro parametri disponibili sono stati assegnati:

**Parametro 1** - controlla la **frequenza** dell'oscillatore: maggiore il valore del parametro, maggiore la frequenza. Il range disponibile va da 0.1 Hz a 20 Hz.



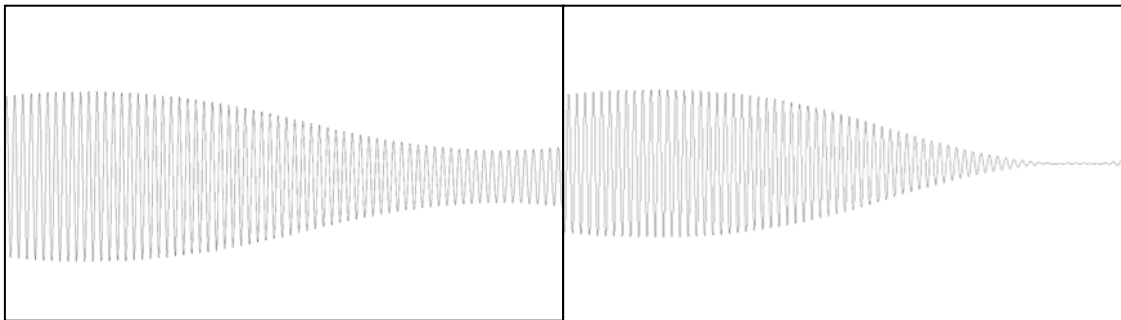
*Fig. 4.18, 4.19 - Segnali modulati da un'onda a rampa a frequenze diverse*

**Parametro 2** - gestisce la **forma** dell'onda generata. Sono selezionabili le 5 forme d'onda descritte nella figura 20.



*Fig. 4.20, 4.21 - Segnale modulato da un'onda a dente di sega (sinistra) e da un'onda sinusoidale (destra). La forma arrotondata del dente di sega è dovuta al filtro passa basso discusso sopra.*

**Parametro 3** - controlla la **profondità** della modulazione. La profondità definisce quanto il segnale di input venga modulato dall'LFO: con la profondità a zero il segnale di uscita sarà uguale a quello di entrata, con la profondità al massimo (cioè  $\sim 1$ ) la modulazione sarà massima.



*Fig. 4.22, 4.23 - Segnale modulato da un'onda sinusoidale, la profondità è rispettivamente a metà (sinistra) ed al massimo (destra).*

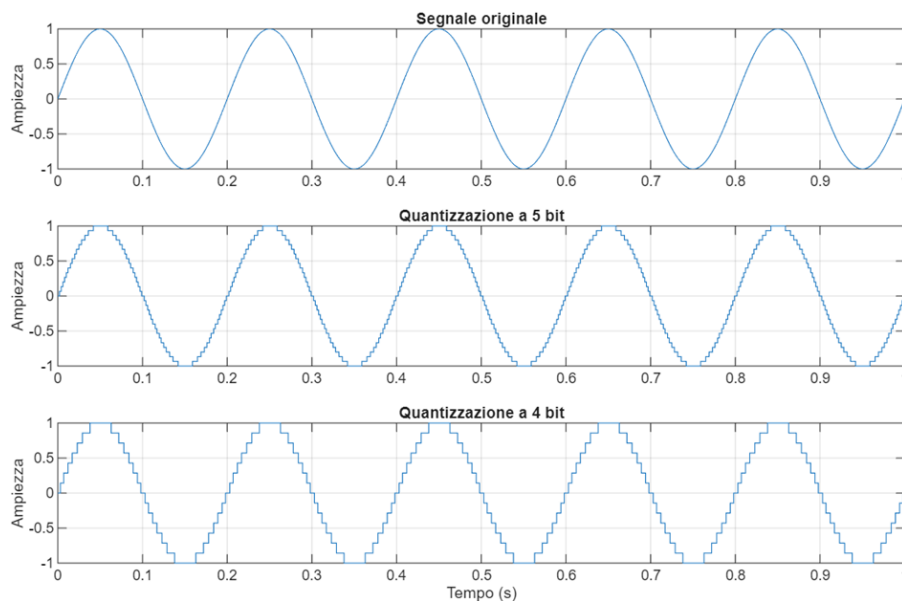
**Parametro 4** - non è assegnato ad alcuna funzione.

L'impiego a regime della CPU si aggira sul 1.85%, mentre la RAM utilizzata è pari a 58.3 KB, cioè il 5.70%.

#### 4.4 - Effetto bitcrusher

Il “bitcrusher” è un effetto relativamente recente, dato che il suo principio di funzionamento nasce dalla digitalizzazione dei segnali. Il suo impiego è emulare sistemi audio con hardware meno potente ed è spesso usato in generi come il “Lo-Fi”. Il suono tipico del bitcrusher proviene da due fenomeni agenti sul segnale: quantizzazione e campionamento ridotto.

La quantizzazione riduce il numero di bit disponibili per descrivere il segnale, diminuendone la risoluzione ed introducendo errori di precisione. Una quantizzazione più aggressiva (quindi un numero di bit di risoluzione minore) rende il suono più distorto e rumoroso.



*Fig. 4.24 - Esempio di quantizzazione a diverse risoluzioni*

La frequenza di campionamento determina invece la fedeltà del suono e l’aliasing delle frequenze (vedi Introduzione): diminuendo la frequenza di campionamento, un numero sempre maggiore di frequenze supereranno la frequenza di Nyquist, perciò si abbasserà la qualità del suono a causa della creazione di “alias”.



Il bitcrusher è implementato dalla classe “bitcrusher\_effect.h”. L’ordine di manipolazione del segnale è: downsampling<sup>6</sup>, quantizzazione, mixaggio con segnale originale ed infine filtraggio. Il downsampling avviene attraverso un algoritmo di “sample and hold”, ovvero un sistema che registra un valore e lo mantiene per un certo numero di cicli. Per il secondo passaggio è usato un quantizzatore uniforme a risoluzione variabile implementato come helper class del bitcrusher, mentre il filtro è lo stesso filtro passa basso usato per la distorsione.

<sup>6</sup> riduzione della frequenza di campionamento

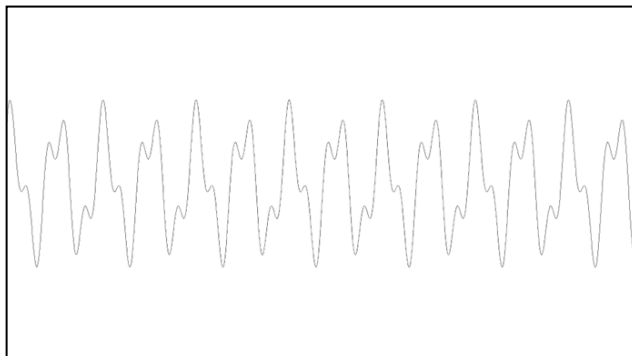
```
void UniformQuantizer::quantize(int16_t &x) {
    bool isNeg = (x < 0);
    int16_t absValue = isNeg ? -x : x;
    uint16_t index = absValue / intervalLength;

    int16_t q = saturate16(index * intervalLength + (
        intervalLength >> 1));

    if(isNeg) {
        q *= -1;
    }

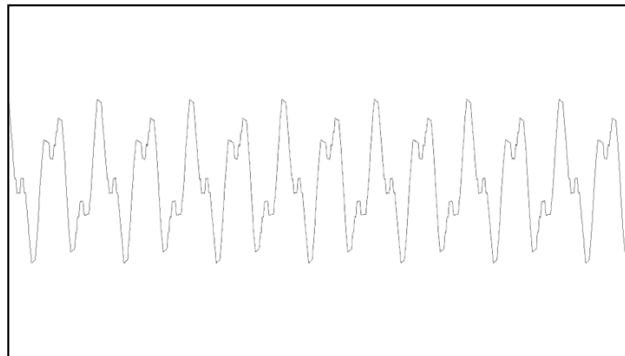
    x = q;
}
```

Il segnale test usato sarà composto da due onde a 200 Hz e 500 Hz sovrapposte:



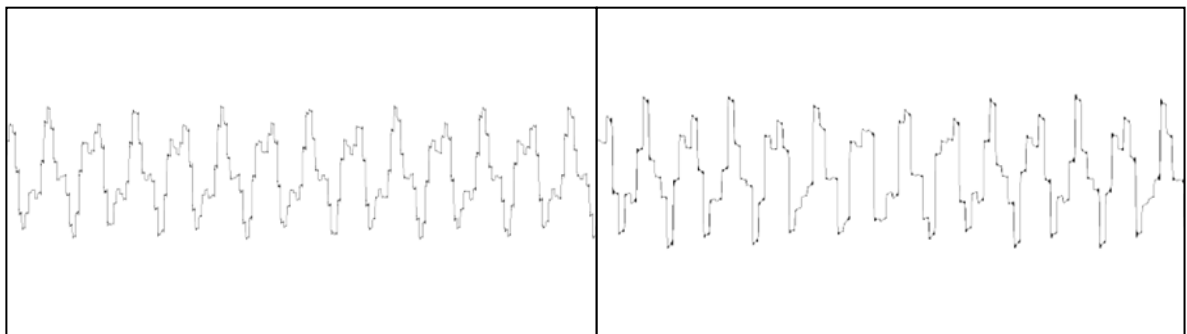
*Fig. 4.25 - Oscillogramma di due onde a 200 Hz e 500 Hz sovrapposte*

**Parametro 1** - controlla la **risoluzione** del quantizzatore. Il range di valori va da 3 bit (forte distorsione del suono e notevole rumore aggiunto) a 14 bit (degradazione del suono quasi impercettibile).

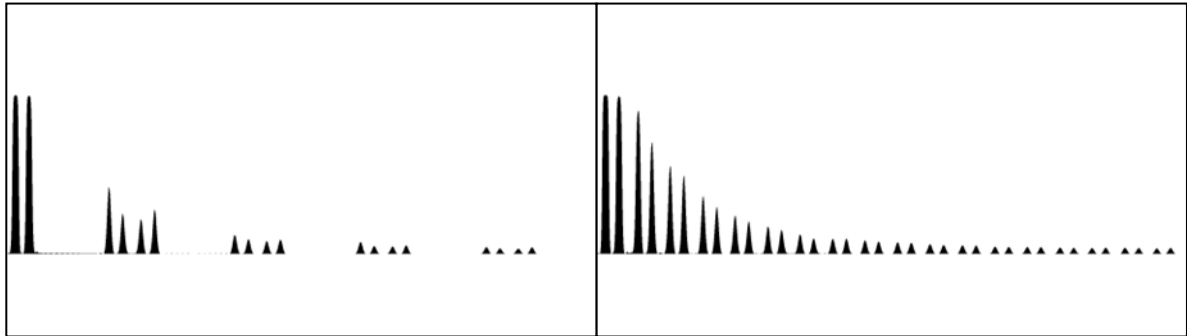


*Fig. 4.26 - Oscillogramma con risoluzione di bit dimezzata (8 bit), l'onda appare molto più "spigolosa" rispetto all'originale poiché sono presenti errori di quantizzazione.*

**Parametro 2** - seleziona il **rapporto** di diminuzione della frequenza di campionamento. Va da 1x, per cui il suono non è modificato, a 32x, per cui la perdita di qualità del suono è molto evidente e sono presenti numerose frequenze di aliasing.



*Fig. 4.27, 4.28 - Oscillogramma con frequenza di campionamento a 1/16, cioè 2.76 kHz (a sinistra) ed a 1/32, ovvero 1.38 kHz (a destra). Nella seconda immagine si può notare chiaramente un numero minore di dettagli nell'onda.*



*Fig. 4.29, 4.30 - Analisi in frequenza con frequenza di sampling a 1/16, 2.76 kHz (a sinistra) ed a 1/32, 1.38 kHz (a destra). Il numero molto maggiore di armoniche nella seconda immagine è dovuto alla frequenza di Nyquist più bassa, che crea molti più alias (si può notare come gli alias sono tutti a coppie come i segnali originali).*

**Parametro 3** - varia il mix di segnale originale (definito “dry”) ed il segnale modificato (detto “wet”).

**Parametro 4** - controlla la frequenza di taglio del filtro passa basso, che va da 1000 Hz a 12 kHz. Con il filtro settato ad 1 kHz il suono è percepibile come particolarmente chiuso e scuro, mentre con l’apertura al massimo il suono contiene molti artefatti.

L’impiego a regime della CPU si aggira sul 2.02%, mentre la RAM utilizzata è pari a 60.1 KB, cioè il 5.87%.

## 4.5 - Effetto vibrato ed effetto chorus

Gli effetti “vibrato” e “chorus” sono effetti che si comportano in modo diverso ma che si originano dallo stesso fenomeno, cioè la modulazione in frequenza.

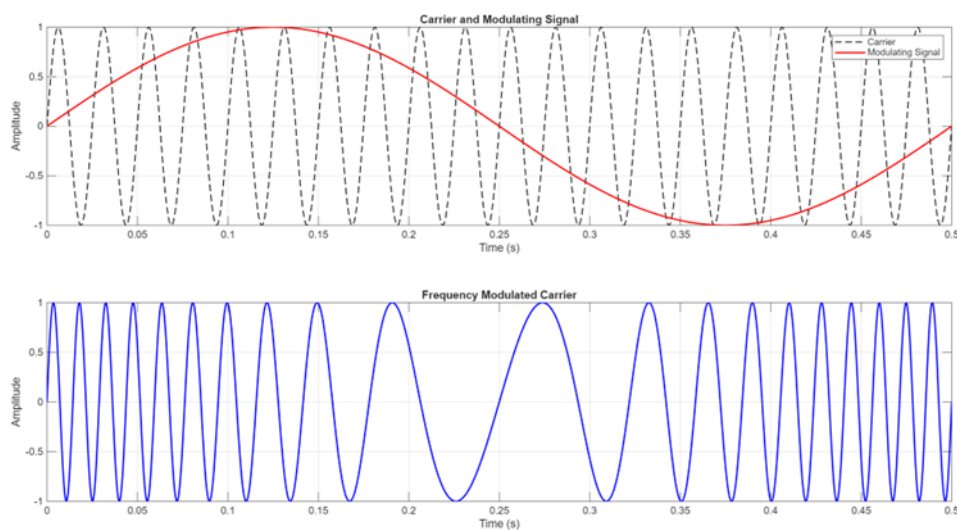


Fig. 4.31 - Esempio di modulazione in frequenza

### 4.5.1 - Effetto vibrato

Il vibrato, ovvero la periodica variazione di intonazione, è un effetto molto conosciuto e viene usato naturalmente come abbellimento nel suono di strumenti e voci<sup>7</sup>, mentre il chorus ha la funzione di imitare la presenza di più strumenti (un coro per l'appunto) intenti a suonare all'unisono ma ognuno con sottili variazioni di intonazione e tempo<sup>8</sup>. Storicamente il primo vibrato è nato negli anni '40 sugli amplificatori Fender, mentre il chorus risale agli organi a canne degli anni '30.

<sup>7</sup> un esempio sono gli strumenti a corda, in cui il vibrato viene generato dai piccoli movimenti periodici della mano sui tasti

<sup>8</sup> alcuni strumenti fanno uso di tecniche equivalenti al chorus: nel mandolino ci sono 8 corde divise in 4 paia con la propria accordatura, ogni coppia di corde è paragonabile ad un chorus a 2 voci su quella frequenza

Il vibrato usa un buffer circolare ed un LFO: il segnale in entrata viene scritto sequenzialmente nel buffer, mentre il segnale in uscita è un campione precedentemente memorizzato, riletto ad una posizione determinata dalla somma di una distanza fissa ed un offset variabile definito dall'LFO. Quando l'offset è positivo la distanza tra gli indici di lettura e scrittura diminuisce, perciò la frequenza è percepita come aumentata (come se il segnale venisse letto più velocemente); quando l'offset è negativo accadrà il contrario, perciò la frequenza diminuirà.

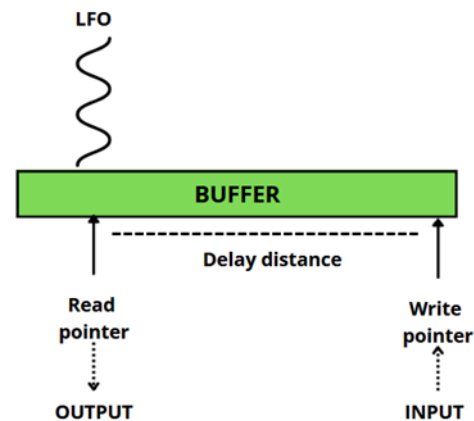


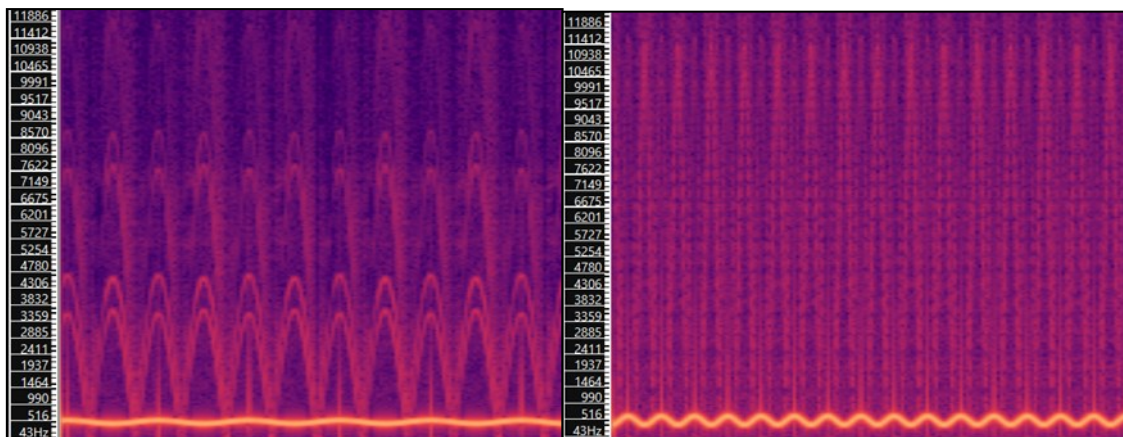
Fig. 4.32 - Schema logico del funzionamento di un vibrato digitale

```
void doVibrato() {
    inputSamplePtr = block->data;
    lfoSamplePtr = lfo_block->data;
    for(int i = 0; i < AUDIO_BLOCK_SAMPLES; i++) {
        writeIndex++;
        if(writeIndex >= VIBRATO_BUFFER_LENGTH)
            writeIndex = 0;
        sampleQueue[writeIndex] = *inputSamplePtr;
        lfoOffset = signed_saturate_rshift(depth *
            (*lfoSamplePtr), 16, 15);
        readIndex = writeIndex - (baseDelay -
            lfoOffset);
        if(readIndex < 0)
            readIndex += VIBRATO_BUFFER_LENGTH;

        lowpassFilter.filter(&sampleQueue
            [readIndex]);
        *inputSamplePtr = sampleQueue[readIndex];
        inputSamplePtr++;
        lfoSamplePtr++;
    }
}
```

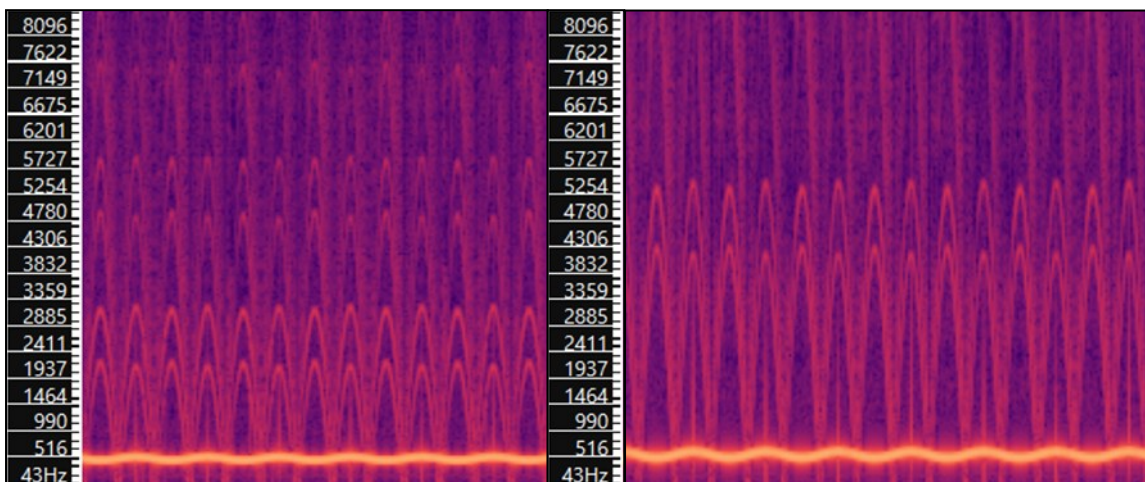
I parametri sono stati assegnati come segue:

**Parametro 1** - regola la velocità di oscillazione dell'LFO. Le frequenze disponibili vanno da 1 Hz a 10 Hz (più basse di quelle di altri effetti come il tremolo).



*Fig. 4.33, 4.34 - Spettrogrammi di 400 Hz modulati con due frequenze diverse. Come si può vedere dalle immagini la trasformazione non lineare del segnale produce armoniche: maggiore è la frequenza di modulazione e più armoniche sono prodotte.*

**Parametro 2** - controlla l'ampiezza massima dell'offset generato: maggiore l'offset, più marcata sarà la “stonatura” imposta al segnale. In termini temporali la **profondità** va da 0 ms a 5 ms.



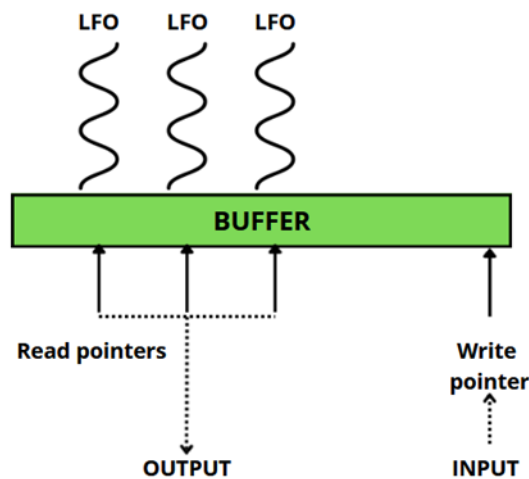
*Fig. 4.35, 4.36 - Spettrogrammi di 400 Hz modulati con due diverse profondità*

**Parametro 3** - gestisce il **tono** del segnale in uscita controllando un filtro passa basso. Oltre a gestire le armoniche prodotte, questo controllo permette di imitare il timbro tipico di un vibrato analogico, caratterizzato da una presenza di frequenze alte attenuata. La frequenza di taglio va da 2kHz a 12kHz.

**Parametro 4** - non è assegnato ad alcuna funzione.

L'impiego a regime della CPU si aggira sul 1.93%, mentre la RAM utilizzata è pari a 59.4 KB, cioè il 5.80%.

#### 5.4.2 - Effetto chorus



Dall'altra parte il chorus impiega lo stesso buffer circolare ma più di un LFO (solitamente fino a 4), ognuno con frequenza e fase leggermente diverse dalle altre. La combinazione dei singoli segnali ritardati e "stonati" crea fenomeni di interferenza costruttiva e distruttiva, simulando l'effetto di un coro di voci. La distanza tra i singoli indici di lettura può variare, ma generalmente la si

tiene ad una costante scelta appositamente per evitare grandi risposte impulsive su determinate frequenze.

*Fig. 4.37 - Schema logico del funzionamento di un chorus digitale*



```
void doChorus() {
    inputSamplePtr = block->data;
    for(int i = 0; i < AUDIO_BLOCK_SAMPLES; i++) {
        writeIndex++;
        if(writeIndex >= CHORUS_BUFFER_LENGTH) {
            writeIndex = 0;
        }
        sampleQueue[writeIndex] = *inputSamplePtr;
        voicesSum = 0;

        //for each voice get the detuned sample
        for(int j = 0; j < voices; j++) {
            lfoOffset = signed_saturate_rshift
                ((int32_t)depth * (*lfoSamplePtrs[j]),
                16, 15);

            int16_t delay = baseDelay - voicesOffset * j
                + lfoOffset;
            readIndex = writeIndex - delay;
            if(readIndex < 0)
                readIndex += CHORUS_BUFFER_LENGTH;
            voicesSum += sampleQueue[readIndex];
        }

        //normalize sum of voices
        *inputSamplePtr = (*inputSamplePtr * (255 -
            mix) + (voicesSum / (voices - 1)) * mix) >> 8;
        inputSamplePtr++;

        for(int k = 0; k < MAX_CHORUS_VOICES; k++) {
            lfoSamplePtrs[k]++;
        }
    }
}
```

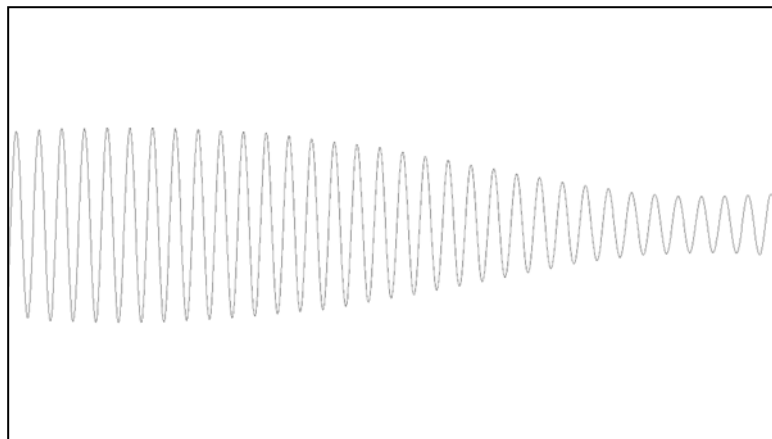
Tutti quanti i parametri controllano un aspetto del chorus:

**Parametro 1, parametro 2** - controllano frequenza e ampiezza dell'onda prodotta dall'LFO (come nel vibrato).

**Parametro 3** - modifica il numero di **voci** usate nella modulazione. Con più voci si avrà un suono più denso e ricco. In questo effetto le voci vanno da 1 a 4.

**Parametro 4** - controlla il mix tra suono originale e suono modulato.

Nota: è facile confondere i concetti di profondità (parametro 2) e numero di voci (parametro 3). La profondità ha l'effetto diretto di rendere evidente la modulazione, dato che controlla quanto viene abbassata o alzata l'intonazione della nota, mentre l'aumentare delle voci ispessisce il suono, facendolo percepire come più "espanso".



*Fig. 4.38 - Segnale a 400 Hz modulato con 4 voci. Sono evidenti sia la variazione di frequenza (il lato sinistro ha frequenza maggiore di quello destro) che la diminuzione di ampiezza dovuta a parziale interferenza distruttiva.*

L'impiego a regime della CPU si aggira sul 2.13%, mentre la RAM utilizzata è pari a 59.7 KB, cioè il 5.83%.



## 4.6 - Caricamento e salvataggio

La classe “SaveManager.h” si occupa della memorizzazione dell’ultima catena di effetti impiegata prima dello spegnimento. Il supporto fisico impiegato è la memoria EEPROM integrata nel microcontrollore, la quale garantisce almeno 100.000 cicli di scrittura per ogni cella. Per semplicità, vengono memorizzati solo gli ID numerici degli effetti, organizzandoli all’interno di uno struct. Poiché la libreria audio non consente di modificare i due capi di un oggetto AudioConnection, le connessioni tra effetti sono allocate dinamicamente dopo il salvataggio dei dati.

```
void storeSettings(Save& s) {  
    EEPROM.put(address, s);  
}  
  
void loadSettings(Save& s) {  
    EEPROM.get(address, s);  
}  
  
struct Save {  
    uint8_t effectIDs[5];  
};
```



## 4.7 - Interfaccia grafica

L'interfaccia grafica è supervisionata dalla classe "ScreenManager.h", che fa uso della libreria open source "U8g2" per gestire lo schermo SSD1309 da 1.54". La connessione avviene con protocollo I2C tramite gli stessi pin con cui il microcontrollore e la scheda audio comunicano, ovvero I2C\_DATA e I2C\_CLOCK. La libreria gestisce la matrice pixel 128x64 tramite una memoria SRAM integrata a cui viene affidato il contenuto di un buffer.

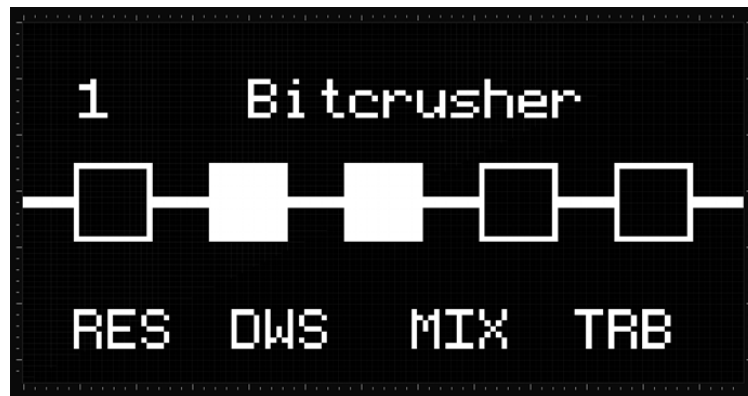
```
void drawEffectNumber(uint8_t index) {
    u8g2.clearBuffer();
    char c[2];
    c[0] = '0' + index;
    c[1] = '\0'; //null-terminated string
    u8g2.setFont(u8g2_font_profont15_mr);
    u8g2.drawStr(effectNumber_padding_x, effectNum
        ber_padding_y, c);
    u8g2.sendBuffer();
}
```

```
void drawIcons(std::vector<bool> &effectsState) {
    u8g2.clearBuffer();
    for(uint8_t i = 0; i < squarePositions.size();
        i++) {
        const uint8_t *icon = effectsState
            [i] ? onIcon : offIcon;

        u8g2.drawXBM(
            squarePositions[i],
            chain_padding_y,
            spriteSize,
            spriteSize,
            icon
        );
    }
    u8g2.sendBuffer();
}
```

L'interfaccia primaria si compone di 4 elementi distinti:

- Posizione dell'effetto selezionato all'interno della catena (in alto a sinistra)
- Nome dell'effetto selezionato (in alto al centro)
- Stato di tutti gli effetti (al centro)
- Nome dei parametri dell'effetto selezionato (in basso al centro)



*Fig. 4.39 - Interfaccia grafica*

Lo stato degli effetti è rappresentato da una “catena” di icone che raffigurano i pedali e le connessioni tra di loro: i quadrati vuoti costituiscono gli effetti spenti (o bypassati), mentre i quadrati pieni quelli attivi. Le immagini sono state dapprima create su GIMP e poi esportate in formato Bitmap; i nomi dei parametri, dato lo spazio ridotto, sono abbreviazioni (es. nell'immagine sopra le abbreviazioni significano “resolution”, “downsampling”, “mix” e “treble”). Ad ogni attivazione, disattivazione o cambio di effetto l'intero schermo è disegnato nuovamente con le informazioni aggiornate.

L'interfaccia secondaria, visualizzata nello stato interno “EDIT”, è composta da due semplici righe di testo, una delle quali mostra il nome dell'effetto selezionato.



## CONCLUSIONE

In questo progetto sono state presentate le fasi di progettazione e implementazione di un sistema modulare di effetti audio, con particolare attenzione all'elaborazione digitale in tempo reale del segnale e alla praticità d'uso in un contesto musicale. Il sistema è stato sviluppato e testato con successo, dimostrando un funzionamento stabile e corretto per tutti i moduli implementati.

Dal punto di vista prestazionale, il sistema garantisce elaborazione in tempo reale affidabile, senza glitch o interruzioni udibili in condizioni operative normali. Gli effetti realizzati presentano un comportamento coerente e controllabile, confermando la validità delle strutture DSP e delle mappature dei parametri.

Particolare attenzione è stata data all'usabilità: ogni aspetto dei parametri e del comportamento di ciascun effetto è stato curato e regolato per assicurare che il sistema interagisca in modo naturale con il segnale di ingresso e l'input dell'utente.

Sono state inoltre considerate le prestazioni in termini di rumore, evidenziando un basso livello di rumore di fondo e l'assenza di artefatti significativi. Ciò indica che sia gli algoritmi di elaborazione che le scelte hardware risultano adeguati per applicazioni audio.

Nel complesso, il progetto soddisfa gli obiettivi prefissati, offrendo una piattaforma compatta e flessibile per la sperimentazione di effetti audio. Possibili sviluppi futuri includono l'introduzione di nuovi effetti, l'ottimizzazione degli algoritmi, l'aggiunta di ulteriori controlli fisici e l'aggiornamento dell'interfaccia utente.



## Bibliografia e sitografia

### Bibliografia

- W. Pirkle, *Designing audio effect plug-ins in c++*, Routledge, 2019
- R. Jaeger e T. Blalock, *Microelectronic circuit design*, Mcgraw-Hill Education, 2010
- A. Clarke, *Digital Implementation of Vibrato*, Sydney University, 2012
- W. Hanson, C. Beagle, A. Altstaetter, A. Patterson, *PolarBear DSP Digital Effects Pedal*, Ohio Northern University, 2025
- B. Holmes, *Guitar Effects-pedal emulation and identification*, Queen's University Belfast, 2019

### Sitografia

- Illumin Magazine (USC), *Pedal to the metal* (<https://illumin.usc.edu/pedal-to-the-metal>)
- [Musicdsp.org](https://www.musicdsp.org/en/latest/Filters/), *Audio-EQ-Cookbook* ([https://www.musicdsp.org/en/latest/](https://www.musicdsp.org/en/latest/Filters/)  
[Filters/](https://www.musicdsp.org/en/latest/Filters/))
- [ElectroSmash.com](https://www.electrosmash.com/pedalshield), *pedalShield Arduino* ([https://www.electrosmash.com/](https://www.electrosmash.com/pedalshield)  
[pedalshield](https://www.electrosmash.com/pedalshield))
- [ElectroSmash.com](https://www.electrosmash.com/tube-screamer-analysis), *Tube Screamer Analysis* ([https://](https://www.electrosmash.com/tube-screamer-analysis)  
[www.electrosmash.com/tube-screamer-analysis](https://www.electrosmash.com/tube-screamer-analysis))
- [ElectroSmash.com](https://www.electrosmash.com/big-muff-pi-analysis), *Big Muff Pi Analysis* ([https://www.electrosmash.com/](https://www.electrosmash.com/big-muff-pi-analysis)  
[big-muff-pi-analysis](https://www.electrosmash.com/big-muff-pi-analysis))
- [ElectroSmash.com](https://www.electrosmash.com/boss-ce-2-analysis), *Boss CE-2 Analysis* ([https://www.electrosmash.com/](https://www.electrosmash.com/boss-ce-2-analysis)  
[boss-ce-2-analysis](https://www.electrosmash.com/boss-ce-2-analysis))